

**context-mode ROI
Analysis — pi-agent-
dashboard**

Date: 2026-07-04 **Question:** Does the context-mode MCP plugin earn its ~24k-token startup cost, or does the project's own kb extension already replace its functions?
Method: Aggregated 1,863 stats-pid-*.json files + tool-call frequency across 623 project session transcripts (443 MB). Schema-cost split parsed from server.bundle.mjs. Analysis run via GLM-5.2 subagent.

Tartalomjegyzék

Verdict: Trim, don't drop

Cost side

Usage data

Sandbox tools (code execution — kb CANNOT replace)

Retrieval tools (kb-like — DO overlap with kb)

kb tools (project's own docs-first retrieval)

Group comparison

Aggregated self-reported savings (1,863 stats files)

Recommendations (ranked by ROI)

(a) Sandbox tools — KEEP

(b) Retrieval tools — DROP (web-fetch aside)

(c) Net verdict — is the ~24k/session tax worth it?

Verdict: Trim, don't drop

The sandbox tools (ctx_execute*) earn their keep — **97% of all context-mode calls** and ~1.87 GB kept out of context, work kb fundamentally **cannot** do (it cannot execute code). But the **retrieval tools are dead weight in this repo**: ~2% of calls, redundant because AGENTS.md *mandates* kb_search first for repo facts and kb is near-free. The ~24k-token/session schema tax is paid by **every** session, yet **79% of sessions never invoke context-mode at all**.

Cost side

context-mode contributes **~24k tokens of tool-schema to every session's startup context**. Its ~40 MCP tools ship verbose WHEN / WHEN-NOT / RETURNS / EXAMPLE descriptions — a fixed per-session tax whether or not the tools are used.

For reference, the full ~51k startup context breaks down as:

Source	~Tokens	Note
Tool definitions (JSON schemas)	~24k	context-mode MCP tools dominate
Base system prompt + guidelines	~9k	identity + harness rules
AGENTS.md (project)	~7k	injected every turn
Memory injections (hermes)	~6.5k	USER / MEMORY / failures / project
Skill catalog (~40 skills)	~4k	name + description only
Session header	~0.3k	id, cwd, date

Usage data

Tool-call frequency across all 623 project transcripts:

Sandbox tools (code execution — kb CANNOT replace)

Tool	Transcript calls
ctx_execute	3,663
ctx_batch_execute	653
ctx_execute_file	111
Total	~4,427

Retrieval tools (kb-like — DO overlap with kb)

Tool	Transcript calls
ctx_search	73
ctx_index	23
ctx_fetch_and_index	12
Total	~108

kb tools (project's own docs-first retrieval)

Tool	Transcript calls
kb_search	78
kb_get	20
kb_neighbors	~0
Total	~98

Retrieval usage is nearly even (context-mode ~108 vs kb ~98), but kb is the project-mandated, near-free tool that owns repo-doc retrieval.

Group comparison

Tool group	Calls	Value	kb overlap	~Schema cost	Recommendation
Sandbox	~97%	1.87 GB	none — kb	~14k tok	KEEP

Tool group	Calls	Value	kb overlap	~Schema cost	Recommendation
(ctx_execute/_file/batch)		kept out of context	can't run code		
Retrieval (ctx_search/index/fetch)	~2%	0.1 MB returned	high — kb owns repo docs	~6k tok	DROP (except web fetch)
Admin (stats/doctor/insight/purge)	<1%	ops only	none	~3.6k tok	keep (cheap)

Aggregated self-reported savings (1,863 stats files)

Metric	Value
Stats files	1,863
Zero-call sessions (paid schema, never used)	1,469 (78.9%)
Bytes sandboxed (summed)	1.87 GB
Bytes indexed (summed)	5.34 MB
tokens_saved_lifetime (max, honest cumulative)	~7.78M tokens
dollars_saved_lifetime (max)	\$69.91
by_tool leader	ctx_execute

Caveat: per-session tokens_saved counts *every* sandboxed byte as “kept out of context” — even bytes the agent would never have read. The summed 469M-token figure is an upper bound, not realized savings. The tokens_saved_lifetime (~7.78M / \$69.91) is the conservative honest metric. Even so, the plugin is net-positive against the cumulative ~44.7M-token schema tax — **but only because the sandbox does the heavy lifting.**

Recommendations (ranked by ROI)

(a) Sandbox tools — KEEP

~4,427 calls (97%), ~1.87 GB kept out of context. kb cannot execute code. This is the value; the ~14k tokens of sandbox schema earn their place.

(b) Retrieval tools — DROP (web-fetch aside)

~108 calls (2%), redundant with the project-mandated kb_search/kb_get. ctx_fetch_and_index (web → markdown → index) is the only non-overlapping capability, and at 12 calls across 623 sessions it is marginal — a curl via bash or a lighter fetch MCP covers it.

(c) Net verdict — is the ~24k/session tax worth it?

Yes, but barely, and only because of the sandbox. Three paths, ranked:

1. **Best ROI — sandbox-only config.** Drops retrieval → sheds ~6k tokens/session (~25%), keeps ~97% of value. **Blocker:** context-mode's config only exposes {enabled: bool} — no per-tool toggle. Requires an upstream PR / one-line patch adding disabledTools.
2. **Pragmatic, zero-effort — default-off, enable on demand.** 79% of sessions never call context-mode. Disable globally; re-enable only in sessions doing heavy log/output analysis. Reclaims the full ~24k for the majority; full power where needed.
3. **Status quo.** Acceptable — sandbox value clears the bar even with redundant retrieval riding along, but leaves ~6k tokens/session on the table.

Drop entirely only if a lighter trusted sandbox/code-exec MCP is already available; otherwise the genuine 1.87 GB-kept-out win is lost. There is no path where keeping the *retrieval* tools is the right call for this repo — kb already owns that lane by mandate.

Numbers sourced by aggregating 1,863 stats-pid-.json files and parsing description lengths from server.bundle.mjs. Self-reported call counts (1,449) are lower than transcript-derived counts (~4,535); the sandbox-dominance ratio (~97%) holds in both populations.*